LAPLACE: A Predictive Execution Runtime for LLM Agents

Anonymous Author(s)

ABSTRACT

Modern Large Language Model (LLM) agents increasingly use the Model Context Protocol (MCP) to interact with external sandboxes. Serving these decoupled services efficiently poses a core architectural tradeoff: keeping all sandboxes hot is too resource-intensive, while on-demand container startup causes cold-start delays that worsen tail latency in multi-tenant deployments. We present LAPLACE, an efficient, low-latency agent serving runtime that unifies streaming prediction and proactive prefetching to provide provisioned-like responsiveness with on-demand resource efficiency. To eliminate invocation latency, LAPLACE uses an asynchronous intent router that runs keyword matching and streaming semantic classification in parallel, detecting sandbox execution intent to deeply overlap LLM inference with sandbox setup. To manage heavyweight sandboxes across workflow steps, LAPLACE employs context-aware stochastic prefetching over a Sandbox Dependency Graph to predict upcoming sandbox transitions and gain prewarm lead time. Furthermore, LAPLACE uses a semantic cache to reuse compatible results and avoid repeated execution, while its out-of-band data bridge uses a shared-memory backplane to bypass the network stack for zero-copy data transmission. Experiments show that, under highly concurrent, multi-turn agent workloads, LAPLACE achieves a 97.9% container prewarming hit rate, reducing P99 end-to-end latency by up to 2.9× compared with the *on-demand* runtime while consuming 45.9% less peak memory than the *provisioned* runtime.

KEYWORDS

LLM Agent, Execution Runtime, Predictive Prewarm

1 INTRODUCTION

Modern LLM agents are evolving from conventional text generators into autonomous systems that iteratively reason, plan, and execute external actions [15, 29, 35]. Unlike traditional LLM serving workloads that primarily optimize token generation [2, 37], agent execution forms a stateful run-loop where an LLM-based controller repeatedly invokes external tools [6, 9, 19, 20] such as code execution, web automation, and data processing services. For security, isolation, and reproducibility, these tools are increasingly deployed as independent sandbox environments and accessed through standardized interfaces such as the Model Context Protocol (MCP) [4]. Consequently, the latency of agent execution is no longer determined solely by model inference, but also by the efficiency of coordinating heterogeneous external execution environments.

To support large-scale concurrent agent sessions, modern cloud platforms increasingly adopt serverless sandbox execution [3, 10, 18], where execution environments are provisioned on demand rather than maintained persistently. This design provides the elasticity required for multi-tenant agent workloads, but introduces a fundamental latency challenge [21, 27, 34, 36]: every tool invocation may incur sandbox initialization overhead, including container image loading, filesystem preparation, namespace configuration, and runtime handshake. These operations can introduce seconds-level delays before actual tool execution begins [11]. In multi-turn agent workflows, such startup overheads accumulate across successive tool invocations, significantly degrading end-to-end responsiveness and limiting the practicality of interactive agent services.

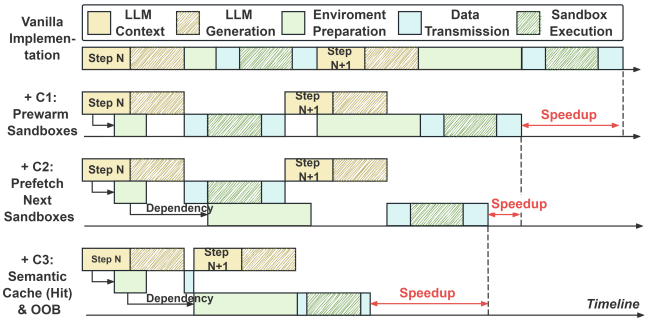


Figure 1: Proactive and overlapped execution v.s. the native vanilla approaches

We identify that the fundamental cause of this latency inefficiency is not sandbox initialization itself, but the execution model adopted by the vanilla implementation of the de facto agent runtimes (e.g., AgentScope[8], AutoGen[33], and LangGraph [14]). These systems follow a reactive and sequential execution model, where sandbox preparation starts only after the LLM completes generation and the tool invocation is fully determined. As a result, the readiness of the environment remains entirely exposed on the critical path, creating repeated idle periods between model reasoning and tool execution. While the GPU is occupied generating tokens, CPU-side resources remain underutilized; once sandbox initialization begins, the GPU often waits for external execution to complete. As illustrated in Fig. 1, this reactive execution model serializes LLM reasoning and sandbox preparation, preventing existing runtimes from effectively overlapping computation with environment provisioning. Consequently, the critical path is repeatedly extended by avoidable infrastructure delays, while both model computation and physical resources remain underutilized at different execution stages. This observation motivates a proactive runtime design that prepares execution resources before they are explicitly requested, enabling latency hiding through cross-stage execution overlap.

Realizing these opportunities raises three challenges. First, early token streams expose only partial semantics, so aggressive intent prediction can activate unnecessary sandboxes and amplify resource consumption. Second, cross-step transitions offer longer prewarm lead time but remain uncertain; prewarming every plausible successor would recreate the resource cost of a provisioned deployment. Third, avoiding repeated execution and large data transfers must preserve the compatibility and visible semantics of existing tool interfaces.

To address these challenges, we present LAPLACE[1], a predictive, proactive serving runtime for efficient multi-tenant LLM agent

[1]Named after *Laplace Transform* and *Laplace's demon*, metaphorizing coordinated execution and predictive orchestration.

workflows. Laplace overlaps LLM execution with sandbox preparation and cross-step workflow execution to break rigid sequential dependencies in vanilla implementations of agent runtime systems. Laplace orchestrates three synergistic components across the control and data planes: i) *Asynchronous Intent Routing*, which operates on streaming token outputs to infer execution intent early and overlap sandbox preparation with ongoing generation within a step; ii) *Stochastic Sandbox Prefetching*, which uses historical agent traces to predict future sandbox needs and pre-warm environments across workflow boundaries, reducing cold-starts to lightweight initialization; and iii) an *Out-of-Band Data Transmission* engine that separates control signals from large execution artifacts, enabling efficient data transmission via a lightweight path that bypasses standard request-response serialization overhead.

Laplace's prototype is implemented and integrated with the open-source AgentScope framework [8]. Laplace is framework-agnostic and highly extensible to any LLM agent serving infrastructures. On diverse multi-turn, high-concurrency, bursty agent benchmarks, Laplace outperforms state-of-the-art serverless runtimes while preserving workflow correctness and protocol compatibility. With a 97.9% container prewarming hit rate, Laplace reduces P99 tail latency by up to 2.9× and peak host memory usage by 45.9%.

This paper makes the following key contributions.

- characterizing the latency structure of multi-turn LLM agent execution and proposing *Asynchronous Intent Routing* (**C1**) to overlap LLM inference with sandbox preparation via inferring streaming token-level intents (§ 3.1).
- devising *Stochastic Sandbox Prefetching*(**C2**) to reduce cross-step cold-start latency by mining historical agent execution traces and pre-warming likely sandbox environments over a Sandbox Dependency Graph (§ 3.2).
- introducing a data-plane optimization framework (**C3**) that decouples control and data paths via out-of-band transport and semantic caching, enabling efficient transfer and reuse of intermediate execution artifacts while eliminating redundant sandbox initialization (§ 3.3).

2 BACKGROUND AND MOTIVATION

2.1 Agent and Environment

Large Language Model (LLM) applications have shifted from monolithic, single-turn chatbots to distributed autonomous agents. Instead of running in a self-contained inference runtime, a modern autonomous agent is executed in an event-driven, distributed run-loop manner [15, 29, 35]. A central LLM-backed controller (the *Agent Engine*) orchestrates the execution by repeatedly reasoning, planning, and invoking external tools. In the context of this paper, we use *turn* to represent a user-visible dialogue round and *step* for a single internal agent iteration within the serving runtime. Our system analysis is performed at the step level.

The execution of agents at runtime is increasingly decoupled from the inference engine. Tasks that require file system interaction, code execution, web access, or queries against proprietary databases are delegated to isolated external execution layers (the *Environment* or *Tool Sandboxes*). Standardized protocols—most notably the Model Context Protocol (MCP) [4] proposed by Anthropic—formalize this interface boundary. MCP specifies a common communication layer over transports such as HTTP and Server-Sent Events (SSE), thereby transforming tool invocation into a set of loosely coupled microservices. This architecture is closely aligned with emerging agent-oriented operating systems that conceptualize LLMs as central processing units and external environments as peripheral devices [8, 17]. From a systems-engineering standpoint, an agent's execution increasingly manifests as a continuous stream of heterogeneous, RPC-like interactions between the LLM inference core and multiple environment services, rather than as a monolithic, localized compute workload.

2.2 Serving Runtimes

Disaggregating Agent Engines from their underlying execution environments enables substantial scalability and architectural flexibility. However, deploying these decoupled components within cloud-native, multi-tenant cluster infrastructures introduces significant challenges related to performance and resource management. Service providers must carefully balance isolation guarantees, resource efficiency, and end-to-end latency, thereby exposing an inherent trade-off between long-running (stateful) and serverless (on-demand) serving paradigms.

Provisioned Runtime. A straightforward strategy [11, 28] is to maintain pre-initialized, always-on containers for each user tenant and its associated sandboxes. However, in contemporary agent ecosystems comprising thousands of fine-grained tools, sustaining all execution environments in a warmed state becomes prohibitively resource-intensive: idle containers incur substantial host memory and CPU overheads [27, 34], which in turn lead to pronounced cluster underutilization and resource interference, thereby rendering large-scale multi-tenant deployments economically unsustainable.

On-demand Runtime. To enhance resource utilization, modern cloud-native platforms commonly employ on-demand runtime provisioning [7, 24], wherein tool sandboxes are instantiated ("cold-started") only upon explicit requests from agents. However, this design choice introduces substantial tail latency: the on-demand initialization of an isolated container incurs a series of serialized overheads, including container image download and extraction, network namespace setup, virtual file system mounting, and application-level handshake procedures. Under bursty workloads or in complex multi-turn workflows, these multi-second cold-start delays accumulate along the agent's end-to-end critical path, thereby inflating P99 tail latency and degrading the interactive quality of service (QoS) of real-time intelligent agents.

2.3 Execution Bottlenecks in Agent Runtimes

In contrast to conventional LLM serving, an agent session constitutes a continuous, stateful execution loop composed of multiple iterative reasoning and sandboxed execution steps. Prevailing agent frameworks employ a purely reactive execution paradigm, in which each tool invocation is initiated only after the LLM has completed its reasoning step. This design induces a strictly serialized dependency chain between model inference and environment interaction. The resulting execution workflow is depicted in Figure 1.

Specifically, we model the end-to-end latency of a single execution step N as:

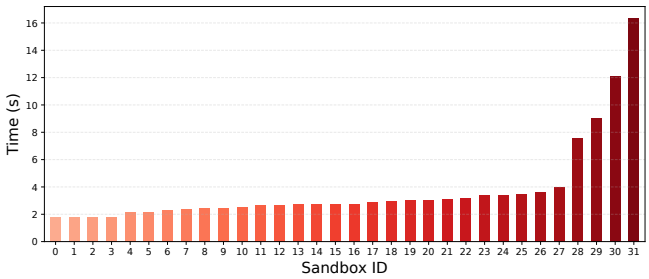


Figure 2: Mean cold-start latency measured across 32 sandboxed environments, encompassing all MCPBench [30] sandboxes as well as additional commonly used environments. The majority of sandboxes exhibit initialization times in the range of 2–4 seconds, whereas more resource-intensive environments require over 4 seconds and can reach up to approximately 20 seconds.

$$T_{step}^{(N)} = T_{context}^{(N)} + T_{generation}^{(N)} + T_{env_readiness}^{(N)} + T_{data_io}^{(N)} + T_{env_exec}^{(N)} \quad (1)$$

The first two components are associated with LLM inference, including context processing and token generation. The remaining three define the optimization boundary for an agent runtime: $T_{env_readiness}^{(N)}$ is the time to make the selected environment ready, $T_{data_io}^{(N)}$ is the time to exchange invocation inputs and results, and $T_{env_exec}^{(N)}$ is the time spent executing the tool.

Laplace targets these time frames without changing the LLM inference stack by overlapping readiness with reasoning, reducing data movement, and eliminating redundant executions. We derived the following observations that motivate this study.

Observation #1: *In-step readiness can overlap with token generation.* In today's reactive execution model, environment initialization cannot start until the LLM emits a full tool invocation, even though the prompt, plan, and partial token stream often already reveal the likely tool. Using this partial evidence to start preparation earlier can put the prewarm even forward, ahead of the corresponding tool invocation. This interval lets sandbox startup, runtime connection setup, and other readiness work overlap with the remaining $T_{generation}^{(N)}$, rather than placing all of $T_{env_readiness}^{(N)}$ on the critical path.As shown in Fig. 2, sandbox initialization frequently requires several seconds and can approach 20 seconds for heavyweight environments, rendering this overlap mechanism critical for minimizing the effective readiness latency observed by the system.

Observation #2: *Cross-step execution can advance the prewarming*. Consecutive steps within an agent session frequently exhibit strong temporal locality. For instance, in a large language model (LLM) serving workload, a `paper search` step is often followed by a `document reading` step that processes a retrieved document, and a `data analysis` step is commonly followed by a `figure generation` step. Nonetheless, prevailing runtime systems generally treat each invocation as an independent event, discarding workflow context once a step is initiated. In contrast, the runtime can exploit the execution window of the current sandbox to proactively instantiate and prewarm the most likely subsequent execution environments before step $N+1$ begins, thereby making the intra-step prewarm more in-advance beyond the current decoding stage.

Observation #3: *Redundant execution and large data movement unnecessarily extend the critical path.* In multi-turn agent workflows, later reasoning steps often revisit previously generated information, such as querying the same document or repeating deterministic analysis. However, conventional runtimes execute these requests again even when a compatible result already exists, repeatedly incurring $T_{env_exec}^{(N)}$. Meanwhile, sandbox interfaces synchronously transfer large intermediate artifacts (e.g., files, images, and structured outputs) with control messages, causing $T_{data_io}^{(N)}$ to grow with artifact size. Reusing compatible execution results and separating artifact delivery from control signaling can therefore remove redundant execution and reduce data movement overhead from the critical path.

These observations reveal optimization opportunities at three points in the agent execution loop: overlapping the environment preparation with the LLM operations within a step, advance the environment preparation of the next step in the current step, and avoiding repeated execution or data movement. Existing LLM serving frameworks primarily focus on optimizing model execution — such as prefill and decoding phases, as well as KV cache management — while serverless runtimes primarily reduce the overhead associated with container initialization without considering user-wise intent. However, neither of them directly addresses how an agent-oriented runtime can leverage these optimizations while maintaining resource efficiency and preserving the intended semantics of tool invocation and usage.

2.4 Research Challenges

Turning each opportunity into a runtime mechanism creates challenges: a practical serving agent must decide early without wasting resources, carry those decisions through uncertain workflows, and optimize data paths without changing visible tool semantics.

Challenge #1: *Early intent prediction vs. resource amplification.* Realizing Observation 1 requires predicting a tool intent from an incomplete token stream. Early-stage tokens are highly ambiguous, especially when general-purpose verbs (e.g., `read`, `get`) are reused across different Model Context Protocol (MCP) tools. Premature prediction can therefore activate the wrong sandbox, amplifying CPU and memory consumption and potentially triggering host-level OOM in multi-tenant deployments. The runtime must obtain enough prewarm lead time to hide readiness while keeping false-positive activations within a bounded resource budget.

Challenge #2: *Cross-step prewarm lead time vs. workflow uncertainty.* Realizing Observation 2 requires selecting next-step sandboxes before the next tool intent is visible. Although workflow transitions are often regular, they are probabilistic and can have multiple plausible successors. Prewarming every successor recreates the resource cost of a provisioned deployment, whereas a conservative prediction loses the longer cross-step prewarm lead time needed to mask second-level sandbox initialization. The runtime must therefore exploit transition history to rank a bounded set of likely environments and retain the current-step signal as a correction mechanism.

Challenge #3: *Reuse and data-path efficiency vs. tool compatibility.* Realizing Observation 3 requires bypassing work without altering what the agent observes from a tool invocation. Exact reuse is safe but misses equivalent requests expressed differently, while unconstrained semantic reuse can return an incompatible result. Likewise,

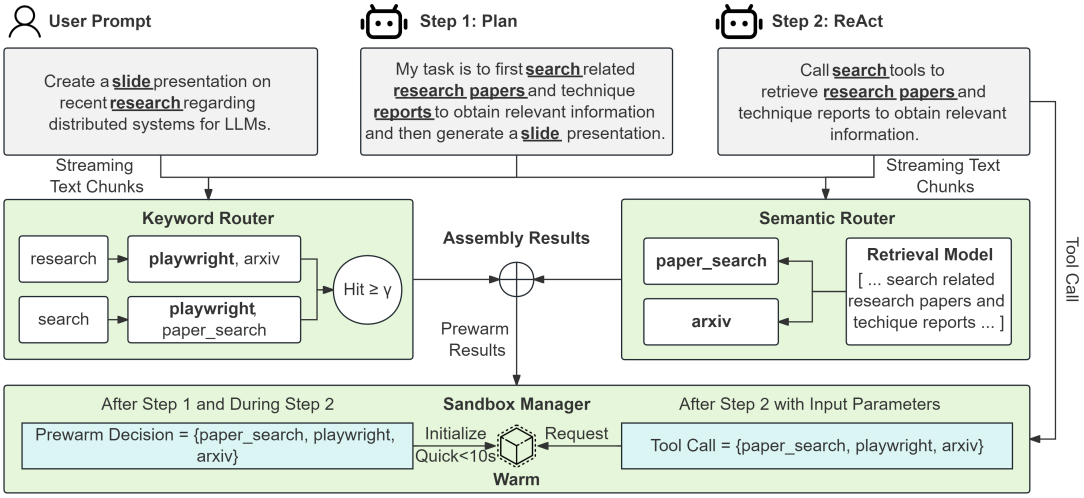


Figure 3: Asynchronous intent routing in LAPLACE. The plan agent expands a user request into ReAct steps; lexical and semantic signals jointly produce a sandbox prewarm set.

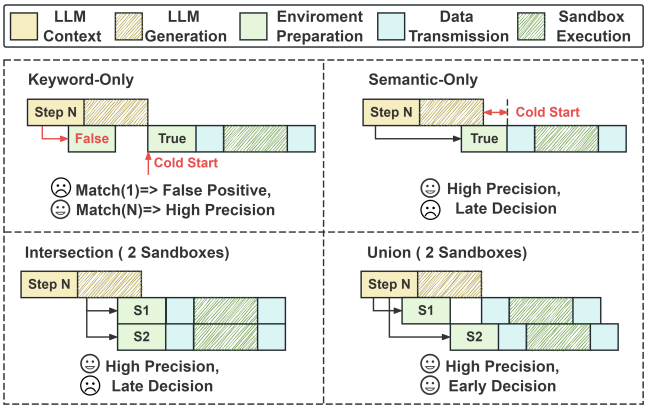


Figure 4: Tradeoffs among routing policies. Keyword routing trades early decisions for potential false positives; semantic routing and intersection delay a precise decision; union assembly retains precision while allowing either router to trigger prewarming early.

embedding large artifacts in ordinary RPC messages preserves compatibility but keeps data movement proportional to payload size; replacing the control interface entirely would disrupt existing tools. The runtime must distinguish compatible reuse from unsafe approximation and separate bulky artifacts from control signaling while preserving visible tool-execution semantics.

3 DESIGN OF LAPLACE

Section 2 identifies three runtime intervals in reactive agent execution: environment readiness, data movement, and sandbox execution. LAPLACE decouples them with three mechanisms. Asynchronous Intent Routing infers an in-step tool intent early to overlap readiness with decoding; Stochastic Sandbox Prefetching exploits cross-step workflow regularity to extend this overlap; and Reuse-Aware Data Transmission avoids redundant execution and removes large artifacts from the control path. This section presents the principal design insights underlying these mechanisms, followed by a discussion of the considerations involved in implementing a runtime system in Section 4.

3.1 Asynchronous Intent Routing (AIR)

At step N, the naive reactive runtime waits for a complete tool invocation before preparing its sandbox, even though the prompt, evolving plan, and partial generation already indicate intent. LAPLACE leverages this interval by streaming context to two independent routers. The design tension is that earlier signals overlap more with $T^{(N)}_{generation}$ but are less reliable and can waste sandbox capacity (Fig. 4). Keyword Router acts early but must balance false positives against a stricter threshold; Semantic Router is more precise but slower. Requiring both signals incurs the Semantic Router's delay, whereas opportunistically combining their outputs preserves high precision while allowing earlier decisions. Hence, at the core of LAPLACE is adopting an asynchronous intent routing mechanism that combines the two asynchronous signals, rather than considering either one on its own to be sufficient.

Keyword Router. The Keyword Router scans the stream against tool-specific keyword profiles. It can emit a candidate within microseconds of a distinctive token, enabling environment readiness while the LLM continues decoding. Common terms like `research`, `search`, or `slide` appear in multiple tool descriptions: triggering on a single match yields a large prewarm set and many false-positive activations, whereas requiring many matches delays triggering past most of the useful overlap window. We therefore apply a threshold γ to the number of matched tool-specific keywords. In Fig. 3, surface cues in the user request and evolving plan let the Keyword Router emit likely sandbox candidates before the plan agent finishes the current ReAct step. The threshold is set to balance early activation with resource amplification; the chosen configuration is given in Section 5.3.

Semantic Router. In parallel, the Semantic Router compares the active context with tool-intent representations. It captures requests whose wording shares little surface vocabulary with a tool profile and disambiguates generic keyword cues using the plan and the preceding generation. In Figure 3, the Semantic Router can identify candidate sandboxes from the broader task intent even when an individual token does not uniquely identify a tool. Its tradeoff is

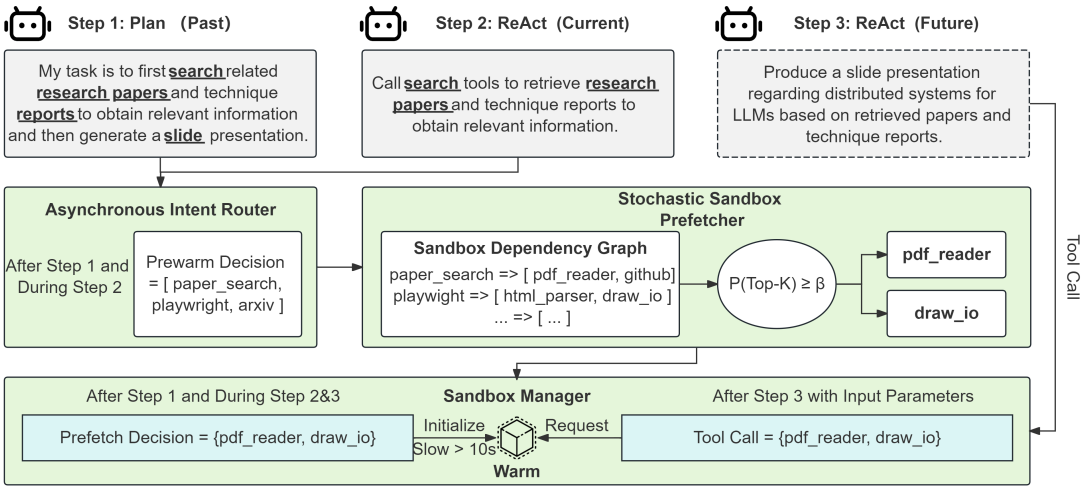


Figure 5: Stochastic sandbox prefetching in LAPLACE. Execution history predicts likely sandboxes for subsequent ReAct steps.

temporal: reliable semantic evidence normally requires a longer prefix, so a semantic-only decision often arrives too late to hide a heavyweight sandbox cold start. The Semantic Router is therefore not a confirmation stage for the Keyword Router. It is an asynchronous, complementary source of candidates that can recover intents the Keyword Router cannot recognize.

Union Assembly. Let $\mathcal{S}^{(N)}_{key}$ and $\mathcal{S}^{(N)}_{semantic}$ denote the candidate sets produced by the two routers at token step N. The lower-left quadrant of Figure 4 illustrates why a conservative design that prewarms only their intersection is not suitable: although intersection improves apparent precision, it delays every trigger until the Semantic Router responds and misses valid tools whenever either router has an imperfect recall. Instead, AIR adopts the lower-right policy and assembles the trigger set as:

$$\mathcal{S}^{(N)}_{trigger} = \mathcal{S}^{(N)}_{key} \cup \mathcal{S}^{(N)}_{semantic} \tag{2}$$

Each router controls its own false positives at its chosen operating point, while the union allows the first credible signal to initiate preparation. Figure 3 shows the resulting execution: the two routers independently derive candidates from the request and the partial plan, then assemble a unified prewarm set for the sandbox manager. Thus, explicit intents can exploit the Keyword Router's early prewarm lead time, and implicit intents can still be captured by the Semantic Router. As will be shown in Section 5.3, this asynchronous union achieves the lowest waiting time and eliminates cold starts on the evaluated routing workload.

3.2 Stochastic Sandbox Prefetching

AIR can only use the remaining decoding time of the current step, which is inadequate when a non-resident sandbox takes longer to start than the marginal token-generation window. Yet agent workflows often repeat tool transitions – for instance, after a paper search, an agent may read a returned document; after data analysis, it may generate a figure or report. LAPLACE exploits these cross-step patterns to start the environment preparation in advance during $T^{(N)}_{env_exec}$, before the next step commits to a specific invocation.

Stochastic Markov Process Modeling. We formulate the steps within an agent workflow as a stochastic Markov process. FIX: LAPLACE records observed sandbox transitions in a *Sandbox Dependency Graph (SDG)*, where a vertex represents a sandbox type and a directed edge records two consecutive tool uses. For the current sandbox v_i, the probability of selecting v_j at the next step is estimated by a first-order transition model:

$$\mathbf{P}_{i,j} = \mathcal{P}\left(S^{(N+1)} = v_j \mid S^{(N)} = v_i\right) = \frac{Count(v_i \rightarrow v_j)}{\sum_k Count(v_i \rightarrow v_k)}. \tag{3}$$

Transition-aware Prefetching. LAPLACE ranks successors by $\mathbf{P}_{i,j}$ and prewarms only those above a probability threshold, subject to a bounded prefetch budget. This budget is crucial: prewarming all successors would approach the cost of full provisioning, while an overly strict policy wastes potential lead time between steps. The first-order model keeps this decision lightweight and off the inference critical path. FIX:clarify first-ordel model. The prefetching mechanism augments rather than replaces AIR. Actually, the SDG provides a workflow before the next intent is known, and AIR then responds to the actual token stream. If predictions are accurate, the target sandbox is already resident before step $N+1$; Section 5.3 will quantify the resulting reduction in late-step cold starts.

3.3 Reuse-Aware Data Transmission

Once an invocation is ready to run, its remaining critical-path cost consists of moving artifacts ($T^{(N)}_{data_io}$) and executing the tool ($T^{(N)}_{env_exec}$). These costs share one execution flow: LAPLACE first determines whether the invocation can reuse an existing result; a cache miss executes the tool and delivers its artifact through a separate data path. Figure 6 illustrates this flow.

Semantic Cache. Multi-turn agents frequently revisit unchanged documents, issue equivalent queries with different wording, or repeat deterministic analyses. Exact argument matching is safe, but it misses these surface-level reformulations. Conversely, unrestricted semantic matching may confuse requests with different tools, inputs, or side effects. The cache must therefore expand reusable requests without relaxing functional compatibility.

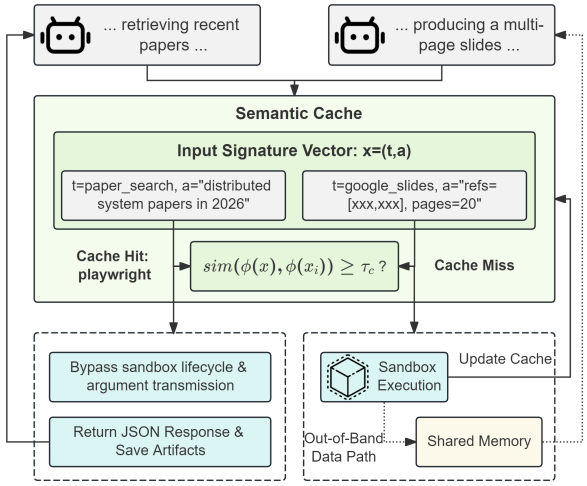


Figure 6: Reuse-aware data transmission in Laplace. A semantic-cache hit returns a prior artifact; a miss executes the tool and delivers the result through the out-of-band data path.

For each completed deterministic invocation, Laplace stores a normalized invocation signature and its result. A lookup first restricts candidates to the same tool identity, then compares the request representation with cached signatures. Given request x, reuse is allowed only when a compatible entry exceeds a similarity threshold:

$$\text{hit}(x) = \exists i \text{ s.t. } tool(x) = tool(x_i) \ \wedge \ sim(\phi(x), \phi(x_i)) \geq \tau_c. \quad (4)$$

This ordering makes semantic matching an extension of exact reuse, rather than a replacement for tool-level compatibility. A miss follows the normal execution path and appends its completed result to the cache. A hit bypasses both sandbox preparation and tool execution, reducing $T^{(N)}_{env_readiness}$ and $T^{(N)}_{env_exec}$. Section 5.3.3 shows that this captures more repeated work than exact matching while preserving a conservative fallback path.

Out-of-Band Data Transmission. Even after a sandbox becomes ready, conventional RPC transports serialize large logs, files, images, and structured outputs into request-response messages. This makes $T^{(N)}_{data_io}$ proportional to payload size and stalls the agent before its next reasoning step. Replacing the control protocol entirely would undermine compatibility with existing MCP tools, so Laplace instead separates control signaling from artifact delivery.

The in-band control channel retains ordinary tool directives, completion notifications, errors, and compact metadata. Large artifacts are represented in that channel only by a fixed-size reference and are exchanged through a co-located zero-copy data path. Accordingly, a cache miss executes normally, publishes its artifact once, and returns a reference to the Agent Engine; a cache hit returns the previously published artifact through that same path. This separation preserves the tool-facing control interface while avoiding RPC serialization and copying for bulky results. Section 5.3.4 shows that the resulting transmission latency is nearly insensitive to payload size.

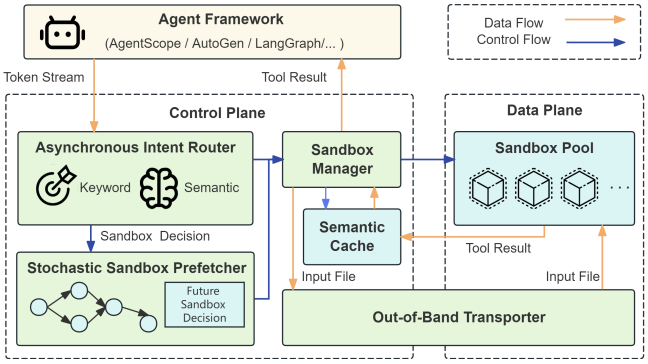


Figure 7: Overview of Laplace.

4 IMPLEMENTATION

Laplace is implemented as a predictive serving runtime between the agent engine and external execution sandboxes. Figure 7 shows its control and data paths. The implementation observes an agent run-loop without changing its tool-facing interface, and overlaps runtime work with the LLM and sandbox work already in progress.

Control Plane. On the control plane, a controller subscribes to the incoming token stream and records completed tool transitions. The Keyword Router matches the stream against tool keyword profiles and emits a candidate when the number of matched tool-specific keywords reaches $\gamma = 2$, the operating point selected in Section 5.3. In parallel, the Semantic Router uses the sparse retrieval configuration selected in that section. Union Assembly dispatches their candidate sets independently to the sandbox manager, which deduplicates requests and starts the corresponding containers. A background prefetch worker updates the Sandbox Dependency Graph from execution traces, applies the transition probabilities in Section 3.2, and submits budgeted next-step warmups while the current tool is executing.

Data Path. Before scheduling a deterministic invocation, the runtime normalizes its arguments, restricts lookup to the same tool identity, and checks its semantic-result cache. Each cache entry contains the tool identifier, normalized arguments, invocation embedding, and result reference; the cache uses $\tau_c = 0.8$ in the reported experiments. On a miss, the sandbox writes a large result into a host-managed memory-mapped shared-memory region. The control channel carries only a 64-bit `token_id` that names this region, while the Agent Engine reads the artifact directly from the shared-memory backplane. Small directives, metadata, completion notifications, and errors remain on the normal control channel. This implements the common cache-or-execute flow in Figure 6 without serializing bulky artifacts through the RPC stack.

Execution boundary and correctness. We consider multi-tenant agents whose tools run in OS-isolated containers or microVMs. Predictive preparation makes an environment ready but does not execute side-effecting work before the agent commits the invocation; unused warmups are discarded. Cache reuse is limited to deterministic, compatible tool requests. The shared-memory bridge operates inside one trusted host or securely managed cluster boundary, with existing access controls preventing cross-tenant memory

access. Sandbox escapes, malicious agents, and compromised infrastructure are outside this work's threat model.

5 EVALUATION

5.1 Experiment Setup

Hardware and Software Environment. All experiments are conducted on a commodity server equipped with an 16-core CPU, 256 GiB of host memory, and a 2 TB NVMe SSD. On the software plane, Laplace is built upon the open-source AgentScope [8] framework, specifically integrated with its agent engine layer. The isolated multi-tenant execution sandboxes are instantiated via Docker containers, and the entire runtime infrastructure is implemented in Python. To drive agent reasoning, we utilize Alibaba DashScope's Qwen3.5-Max model, accessed concurrently via its production cloud API endpoints.

Workloads. We evaluate Laplace using a trace-level benchmark derived from MCPBench [30] and 32 MCP-compatible tool servers collected from GitHub [9], including Playwright [19], Jupyter [6], and Neo4j [20]. These tools span representative execution domains in modern LLM agent workflows. The benchmark is constructed via a two-stage synthesis pipeline. In the first stage, we generate single-turn interaction templates for each tool server, covering all 32 tools with 20 distinct natural language requests per tool, forming a diverse set of atomic tool-use behaviors. In the second stage, an LLM-based planner composes these atomic interactions into coherent multi-turn execution trajectories by reasoning over tool semantics and workflow dependencies, producing 1–10 step agent sessions. This process enables structured task composition such as sequential operations (e.g., search literature followed by slide generation) while preserving tool-level validity and diversity.

Overall, we generate 200 multi-turn traces, uniformly distributed across trajectory lengths (1–10 steps, 20 traces per step), ensuring balanced coverage over all tool servers and reducing tool-specific bias. Each trace represents a complete agent session with sequential tool invocations across heterogeneous execution environments, with an average of 6.4 steps per session and 2.96 tools per step.

Methodology. Unless otherwise specified, we evaluate all systems by replaying full traces as session-level workloads with a fixed random seed (seed = 0). Each trace is executed in a step-wise manner, preserving dependencies across reasoning, tool invocation, and sandbox execution to faithfully model realistic multi-turn agent workflows. Under this deterministic sampling setting, the resulting trace distribution naturally exhibits a concentration in the 5–8 step range, which we therefore treat as the representative workload regime rather than a manually selected subset.

We evaluate all experiments at the session level unless explicitly stated otherwise, and defer workload-specific configurations (e.g., trace length or sampling range) to each individual ablation study.

Evaluation Metrics. We evaluate Laplace across three dimensions aligned with system design goals:

- **Latency:** End-to-end execution latency per agent session, reporting mean and tail latency (P99) to capture both average performance and long-tail behavior under multi-step execution.
- **Resource Efficiency:** We measure peak CPU and memory consumption under multi-tenant workloads to quantify the runtime overhead introduced by sandbox provisioning, execution, and lifecycle management.
- **Prediction Accuracy:** We evaluate the correctness of all predictive runtime optimizations, including asynchronous intent routing, stochastic sandbox prefetching, and semantic cache reuse. We report the accuracy of each mechanism in terms of (i) correct triggering or retrieval decisions under uncertain tool intent, and (ii) false positive rate, capturing unnecessary or incorrect activations such as misrouted intent predictions, redundant sandbox prewarming, or invalid cache hits under semantic ambiguity.

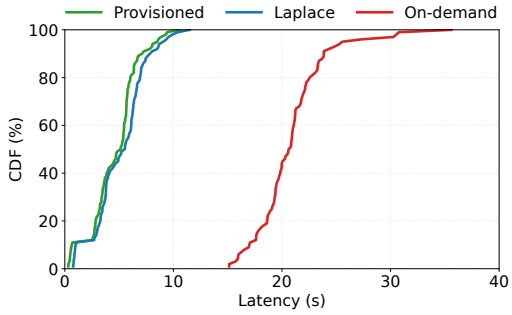


Figure 8: Cumulative tool provisioning latency across multi-turn agent sessions (5–8 steps per session).

5.2 End-to-End Performance

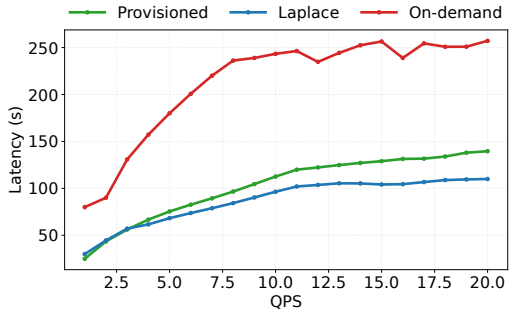
This section evaluates the end-to-end (E2E) performance of Laplace under two representative baselines in agent serving. We define our baselines as follows:

- **Provisioned Runtime:** Optimized for latency by maintaining warm containers, representing the performance upper bound but the resource efficiency lower bound.
- **On-demand Runtime:** Optimized for resource efficiency by instantiating containers per request, representing the cost efficiency upper bound but the latency lower bound.

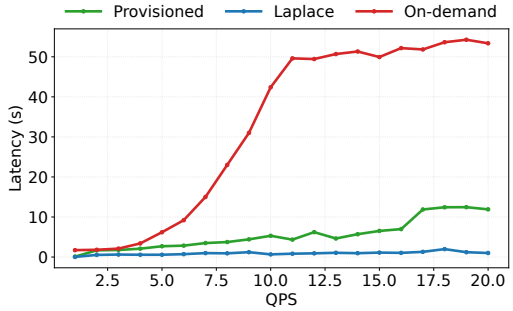
Our evaluation aims to answer a fundamental question: *Can Laplace successfully decouple execution latency from physical resource constraints, achieving serverful-like performance with serverless-like cost?*

5.2.1 Cumulative Tool Provisioning Latency in Multi-Turn Agent Workflows. We evaluate the end-to-end impact of Laplace on tool provisioning latency over multi-turn agent execution traces. We define cumulative tool provisioning latency as the sum of sandbox initialization delays incurred at each tool invocation within a session, focusing exclusively on environment setup overheads and excluding in-sandbox computation time.

Figure 8 shows the distribution of cumulative latency across sessions. The *On-demand* baseline exhibits steadily increasing latency over longer execution horizons due to repeated cold-start overheads, resulting in a heavy-tailed distribution. In contrast, Laplace significantly reduces cumulative latency and maintains a much tighter distribution concentrated in the sub-second range. Compared to the

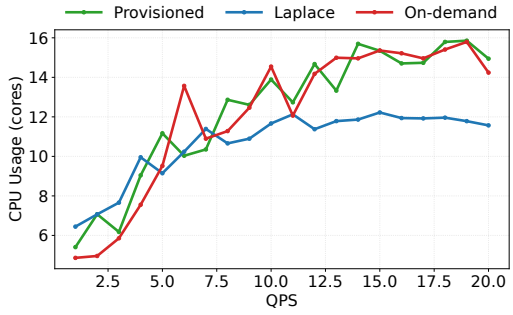


(a) P99 E2E Latency

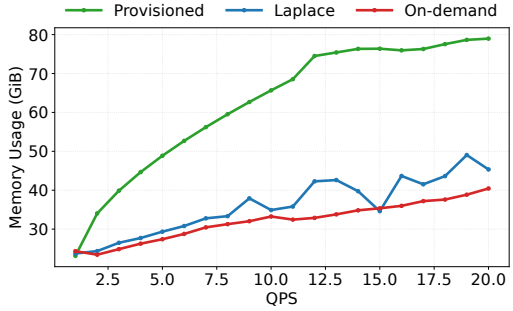


(b) Mean Cumulative Tool Provisioning Latency

Figure 9: End-to-end latency under concurrent loads: (a) P99 E2E latency, and (b) Mean cumulative tool provisioning latency.



(a) Peak CPU Usage



(b) Peak Memory Usage

Figure 10: Physical resource consumption profiles across the load sweep: (a) Peak CPU usage, and (b) Peak memory usage.

On-demand baseline, LAPLACE achieves a 4.53× reduction in cumulative tool provisioning latency. Against the *Provisioned* baseline, LAPLACE remains within a 10.6% performance gap while avoiding the substantial resource overhead of persistent sandbox allocation. These results demonstrate that predictive orchestration can effectively approximate serverful execution performance under a serverless deployment model.

5.2.2 Concurrency Scalability. Figure 9a summarizes the concurrency scalability of the three runtimes as QPS increases from 1 to 20. In the low-to-mid load regime, LAPLACE consistently tracks a latency profile close to *Provisioned* while still reducing end-to-end delay relative to *On-demand*. At higher concurrency, the advantage over *On-demand* remains substantial: at QPS=20, Laplace achieves 88.7s P99 E2E latency, a 2.9× speedup over *On-demand* (257.2s). This indicates that LAPLACE can absorb increasing concurrency without inheriting the long-tail latency explosion of the serverless baseline.

Figure 9b also reveals that the main scalability bottleneck of *On-demand* lies in cumulative tool provisioning. In the low-QPS regime, the mean cumulative tool provisioning latency stays below 4 seconds for all three modes. However, once the load reaches the higher-QPS regime (QPS ≥ 5), *On-demand* suffers growing degradation from network contention and resource constraints, and its cumulative tool provisioning latency rises steadily and eventually surpasses 50 seconds. By contrast, *Laplace* remains tightly bounded in the sub-5-second range and continues to slightly outperform *Provisioned* in tool provisioning latency, reflecting the benefit of LAPLACE's asynchronous intent routing and stochastic sandbox prefetching under concurrency pressure.

5.2.3 Resource Footprint and Efficiency. Figure 10a shows that the CPU footprint of the three runtimes remains relatively compact under low-load conditions but diverges significantly as concurrency scales. While *LAPLACE* exhibits no obvious advantages under low QPS loads, its strengths gradually emerge in the high-QPS regime where QPS ≥ 8. It consistently restrains peak CPU utilization across all tested loads, with the peak resource consumption capped at around 12.2 cores. This translates to a 22.8%–23.3% reduction in peak CPU pressure compared to both *On-demand* and *Provisioned*, which demand roughly 15.8–15.9 cores. Particularly in the high-QPS regime, *LAPLACE* stabilizes within a tight band of ∼ 11–12 cores, whereas the two baselines fluctuate at a higher plateau of 14–16 cores, demonstrating that LAPLACE effectively mitigates CPU contention (saving up to 25% CPU resource under peak load) without sacrificing sub-millisecond control-plane responsiveness.

The memory footprint (Figure 10b) exhibits a pronounced resource separation across the evaluation spectrum. As expected, *Provisioned* is the most memory-intensive variant due to its persistence strategy; its memory consumption surges from an initial 24.6 GiB at light load to a massive peak of 80.6 GiB, staying sustained above 60 GiB as concurrency intensifies. Conversely, *On-demand* maintains a lean profile, bounding its peak usage between 24.3 GiB and 40.4 GiB. Notably, *LAPLACE* closely mirrors the trajectory of *On-demand* across the majority of QPS configurations, topping out at 49.4 GiB. This represents a 45.9% reduction in peak memory footprint compared to *Provisioned*, demonstrating that LAPLACE

Table 1: Keyword threshold sensitivity ($\gamma = 1, 2, 3$).

γ	Avg Wait (ms)	Match Rate	Mismatch Runs
1	786.619	80.0%	20
2	323.021	95.0%	5
3	621.812	97.0%	3

successfully eliminates the prohibitive memory holding costs of serverful runtimes, while maintaining a predictable, serverless-like resource elasticity under heavy concurrent workflows.

5.3 Ablation Study

This section provides mechanism-level attribution for the end-to-end improvements reported above. In particular, we separately isolate the effects of asynchronous intent routing, stochastic sandbox prefetching, semantic caching, and out-of-band data transmission to avoid conflating component contributions in the E2E section.

5.3.1 Effectiveness of Asynchronous Intent Routing. We further decompose Asynchronous Intent Routing into three orthogonal factors: the keyword trigger threshold, the semantic model family, and the hybrid fusion policy. All results are measured on the same sampled tasks and repeated 100 times unless otherwise specified.

Sensitivity of Keyword Threshold γ. To rigorously isolate the impact of the keyword matching threshold, we evaluate $\gamma \in \{1, 2, 3\}$ under identical workload profiles, executing 100 independent trials for each configuration to ensure statistical convergence. As quantified in Table 1, $\gamma = 2$ emerges as the optimal configuration for the Keyword Router. Specifically, $\gamma = 2$ substantially compresses the average keyword-driven waiting latency to 323.0 ms (a 2.43× reduction compared to 786.6 ms under $\gamma = 1$) while maintaining a high predictive routing coverage of 95%.

The severe latency degradation under $\gamma = 1$ is primarily driven by its hyper-sensitivity, which triggers 20 distinct tool mismatch instances across the 100 runs; these false positives inadvertently amplify cold-start fallback penalties and introduce heavy tail latencies. Conversely, while increasing the threshold to $\gamma = 3$ further sharpens routing precision (yielding a 97% match rate with only 3 mismatch instances), it severely erodes the prewarm lead time, causing the average waiting time to climb back to 621.8 ms. This empirical trade-off confirms that $\gamma = 2$ effectively balances early predictive agility with resource stability.

Sensitivity of Semantic Router Models. Table 2 evaluates the performance and computational trade-offs across three distinct structural granularities of semantic embedding pipelines under their respective optimal operating configurations:

- **Retrieval:** This engine is instantiated via a non-neural, sparse token-level retrieval pipeline. It leverages TF-IDF [26] weighting combined with a *Top-N* nearest neighbor aggregation network, constructing composite vector features across token unigrams, selective bigrams, and character n-grams (ranging from 3 to 5 characters).
- **Encoder:** This neural engine is built upon a fine-tuned *all-MiniLM-L6-v2* [23] transformer sequence representation network. It maps fluid multi-turn dialogue histories into dense vector spaces via contrastive pair-wise optimization under a *CosineSimilarityLoss* constraint.
- **FastText:** This lightweight baseline [12] represents text as the average of word and subword embeddings followed by a linear classifier.

Table 2: Semantic router model comparison (best operating point per model).

Model	Micro-F1	Hit Rate	Precision	Avg Latency (ms)
Retrieval	0.970	1.000	0.942	2.116
Encoder	0.776	1.000	0.634	7.822
FastText	0.124	0.124	0.124	0.171

Empirically, as quantified in Table 2, both the *Retrieval* and *Encoder* pipelines achieve a perfect predictive prewarming Hit Rate (1.000), demonstrating that both sparse lexical tokens and dense semantic features can successfully capture all necessary tool invocation targets. However, the sparse *Retrieval* model achieves higher orchestration quality and routing precision, yielding a Micro-F1 of 0.970 and a Precision of 0.9417, whereas the neural *Encoder* baseline degrades to a Micro-F1 of 0.776 and a Precision of 0.634.

Furthermore, from an architectural efficiency perspective, the *Retrieval* model operates with a significantly lower computational footprint, executing with an average inference latency of only 2.116 ms–yielding a 3.70× speedup compared to the 7.822 ms latency incurred by the *Encoder*. Conversely, while the *FastText* pipeline provides ultra-low execution latency (0.171 ms), its shallow token representation space fails to deliver usable discrimination under multi-turn reasoning context drifts, resulting in a Micro-F1 and Hit Rate of merely 0.124. Consequently, the non-neural *Retrieval* pipeline is selected as the production instance for Laplace's semantic branch, as it strikes the optimal Pareto-efficiency frontier between high-fidelity prediction accuracy and low-overhead control-plane latency.

Sensitivity of Hybrid Fusion Policies. To rigorously isolate the algorithmic impact of the intent combination layer, we evaluate three distinct fusion policy paradigms under identical multi-turn context distributions:

- **Union (∪):** The predictive prewarming primitive is non-blocking and asynchronously dispatched the exact microsecond *either* the fast-path lexical tracker *or* the semantic retrieval engine hits their respective individual activation boundaries.
- **Intersection (∩):** Predictive orchestration strictly enforces a dual-engine consensus; a tool container is prewarmed if and only if *both* the lexical scanning and semantic embedding engines concurrently validate the tool candidate's invocation intent.
- **Weighted:** This hybrid policy aggregates multi-modal intent metrics into a centralized candidate score: $s(c) = w_k \cdot s_k(c) + w_s \cdot s_s(c)$, where $s_k(c)$ denotes the matched-keyword ratio (normalized against the static lexicon scale) and $s_s(c)$ represents the real-time semantic retrieval similarity vector. A predictive trigger is dispatched only when clearing a rigid fusion threshold: $s(c) \geq \tau_f$. In our empirical runner, we implement a symmetric baseline configuration with $w_k = 0.5$, $w_s = 0.5$, and $\tau_f = 0.5$.

Table 3: Hybrid fusion policy trade-off.

Fusion Mode	Avg Wait (ms)	Target Match Rate	Cold Start Ratio
Union	124.446	100.0%	0.0%
Intersection	393.523	90.0%	10.0%
Weighted	1308.378	55.0%	45.0%

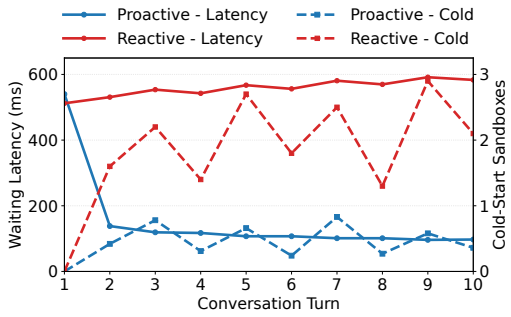


Figure 11: Dynamic execution profiling across a 10-turn conversation horizon, benchmarking per-turn average waiting latency (left) against the corresponding average cold-start sandbox activation count (right).

As quantified in Table 3, the opportunistic *Union* policy achieves the absolute Pareto-optimal performance frontier, securing a flawless 100.0% target-match rate and completely eradicating cold-start events (0.0%), thereby compressing the aggregate average waiting latency to a mere 124.45 ms. Conversely, the conservative *Intersection* policy severely handicaps performance; by strictly blocking for dual-engine consensus, it delays predictive activation on the critical path, driving the latency up to 393.52 ms while introducing a 10.0% cold-start ratio.

The *Weighted* policy exhibits the worst performance degradation within our dynamic runtime context, increasing the average waiting time to 1308.38 ms (a 10.51× increase compared to *Union*). This systemic failure stems from a severe structural misalignment between lexical tokens and high-dimensional semantic embeddings over fluid reasoning horizons. Because the additive score calculation lacks a dynamic dynamic re-normalization denominator ($w_k + w_s$), shifting context distributions under multi-turn reasoning drifts easily prevent the combined score from clearing the static $\tau_f = 0.5$ frontier. This severe match degradation (55.0%) and subsequent cold-start amplification (45.0%) support our architectural choice of a Union-first design philosophy to maximize latency-masking agility without sacrificing routing correctness.

5.3.2 Effectiveness of Stochastic Sandbox Prefetching. To isolate the contribution of stochastic prefetching under multi-turn agent execution, we evaluate Laplace with two deployment variants under an identical planning workload:

- **Laplace-Reactive:** A routing-only baseline that performs inline token-level intent detection but does not use cross-step transition prediction.
- **Laplace-Proactive:** The full design that couples online hybrid routing with a stochastic Markovian prefetcher over the Sandbox Dependency Graph (SDG), enabling asynchronous sandbox preparation before the next step.

Table 4: Semantic cache trade-off.

Setting	Avg Wait (ms)	Hit Rate	Bypass Ratio
No Cache	412.78	0.0%	0.0%
Exact-Match Cache	233.64	33.6%	33.6%
Semantic Cache ($\tau_c = 0.8$)	141.93	37.4%	31.7%

Figure 11 reports per-turn average waiting latency and cold-start counts over a 10-turn horizon. During Turn 1, both variants show similar latency (512.06 ms vs. 540.06 ms), since no historical transition signal is available to initialize the Markov predictor. From Turn 2 onward, the two trajectories diverge sharply. *Laplace-Proactive* reduces average waiting latency from 540.06 ms to 138.14 ms at Turn 2 and further to 97.14 ms by Turn 10, while *Laplace-Reactive* increases to 583.26 ms at Turn 10. This corresponds to a 6.0× end-horizon latency reduction. The mechanism is consistent with the cold-start telemetry. Under *Laplace-Reactive*, average cold-start count peaks at 2.90 instances per turn (Turn 9), indicating repeated exposure to container initialization cost on the critical path. In contrast, *Laplace-Proactive* keeps per-turn cold starts within 0.24–0.83, effectively masking startup latency through early scheduling. These results show that Laplace improves multi-turn responsiveness through accurate, low-overhead temporal prefetching rather than infrastructure over-provisioning.

5.3.3 Effectiveness of Semantic Cache. Semantic Cache targets a different bottleneck from data transport: instead of reducing how long it takes to move a result, it avoids recomputing a result that has already been observed for the same tool and a semantically equivalent request. This matters most in read-heavy workflows where agents repeatedly revisit the same documentation page, rerun the same notebook summary on an unchanged dataset slice, or issue near-duplicate inspection requests after a minor prompt rephrasing.

To quantify this effect, we benchmark three variants under the same repeated-request workload over 100 runs:

- **No Cache:** The baseline that executes every repeated request from scratch.
- **Exact-Match Cache:** A strict cache that matches on tool identity and normalized argument equality.
- **Semantic Cache:** The proposed cache that reuses results when tool identity matches and semantic similarity exceeds the threshold.

We report average waiting latency, cache hit rate, and sandbox bypass ratio.

As shown in Table 4, exact-match caching already provides a measurable improvement over *No Cache*, but semantic caching captures more near-duplicate requests by tolerating surface-form variation. Compared with *No Cache*, semantic caching reduces average waiting latency by 2.91× (412.78 → 141.93 ms) and raises cache hit rate from 33.6% under exact matching to 37.4%. Its sandbox bypass ratio, however, is slightly lower than exact matching because a small fraction of semantic hits are not sufficiently reliable to skip sandbox setup and must fall back to normal execution after validation. This behavior is consistent with the intended design: semantic equivalence expands the reusable request space, but some loose

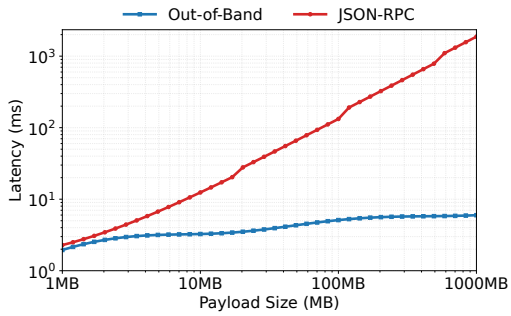


Figure 12: Data transmission latency scaling profiles under varying payload sizes, comparing Laplace's out-of-band zero-copy pipeline against standard JSON-RPC serialization.

matches still require conservative verification before execution can be bypassed.

These results also reinforce the orthogonality of data-plane optimizations in Laplace: out-of-band transmission reduces payload movement cost after execution, whereas semantic caching eliminates redundant execution altogether when reuse is possible.

5.3.4 Effectiveness of Out-of-Band Data Transmission. To evaluate the architectural efficiency of our control-and-data plane separation, we isolate the communication and data-shuffling overheads by benchmarking two distinct transport paradigms across an exponential data payload spectrum scaling from 1.00 MB to 1000.00 MB:

- **Out-of-Band:** Our proposed out-of-band transport mechanism. It completely bypasses the control-plane RPC tunnel by writing bulky state payloads directly to a dedicated local shared-memory substrate or zero-copy virtualized host-guest ring buffers, passing only lightweight, fixed-size references over the wire.
- **JSON-RPC:** The standard baseline paradigm utilized in conventional serverless or agent runtimes. It marshals multimodal data payloads directly into the inline runtime execution stream, forcing the control-plane to serialize and transport raw data matrices synchronously via text-based JSON-RPC network primitives.

As quantified in Figure 12, the empirical measurements reveal a clear scaling divergence between the two protocols across an exponential payload spectrum. At the baseline threshold (1.00 MB), both configurations demonstrate sub-3 millisecond performance, with *Out-of-Band* maintaining a slight edge (1.95 ms vs. 2.28 ms). However, as the payload expands, the *JSON-RPC* pipeline suffers a catastrophic linear performance degeneration; throttled by heavy serialization bottlenecks, its latency grows to 132.45 ms at 100.00 MB and reaches 1873.16 ms at the 1000.00 MB boundary. Conversely, *Out-of-Band* demonstrates a near-constant $O(1)$ scaling profile, drifting to only 5.97 ms at the 1 GB boundary—a 313.55× latency reduction.

The root cause of this performance gap is the complete elimination of critical-path serialization and memory copying penalties. Inside autonomous multi-turn reasoning workflows, agents frequently exchange bulky multi-modal states such as high-dimensional vectors or intermediate media histories. Under the inline paradigm, the control-plane loop stalls because the host CPU must synchronously block to marshal and parse raw text tokens out of network streams. By contrast, *Laplace-OOB* effectively decouples control telemetry from raw payload routing, turning data movement into a size-invariant reference-passing operation over shared-memory substrates. These results show that separating control signals from data payloads keeps Laplace's scheduling overhead at the microsecond level regardless of payload size.

6 DISCUSSION

6.1 Architectural Overhead

A critical concern in predictive serving runtimes is whether the system-level orchestration mechanisms introduce non-negligible processing penalties onto the critical path. In Laplace, this control-plane overhead is thoroughly isolated from the GPU-bound inference loop through strict out-of-band execution and parallel routing mechanics. While the token-level lexical scanning runs with deterministic $O(1)$ complexity, the inherently heavier semantic retrieval engine is offloaded to dedicated background CPU worker threads. Because the hybrid router utilizes an opportunistic *Union* ($\cup$) policy, the critical path never blocks for late-arriving semantic embeddings; any early fast-path trigger instantly dispatches the activation primitive within microseconds. Combined with the prefetching daemon—which evaluates low-dimension Markov matrix transitions bounded by the small cardinality of active tools—Laplace restricts its control-plane telemetry to tens of microseconds, securing a nearly zero-cost latency impact on token generation.

6.2 Ecosystem Generalization

Although our prototype is implemented on top of AgentScope [8], the design of Laplace is not limited to agent serving frameworks. We position Laplace as a runtime middleware layer operating between upstream reasoning systems and downstream execution environments. This design leverages two widely available capabilities in modern agent ecosystems [14, 33]: streaming token-level outputs from upstream frameworks, and standardized tool interfaces provided by protocols such as MCP [4]. As a result, Laplace requires no additional system-level constraints beyond what is already supported in existing agent and tool orchestration stacks.

Importantly, this execution abstraction extends beyond inference-time serving to agentic reinforcement learning (Agentic RL) frameworks, such as VERL [25] and Slime [39], where rollout generation and environment interaction follow a structurally similar loop. In these Agentic RL settings, Laplace 's runtime optimizations—namely asynchronous intent routing and predictive prefetching—can be seamlessly adapted to mask environment initialization and interaction overheads during training rollouts with modest integration effort.

6.3 Limitations and Future Works

While Laplace effectively eliminates massive cold-start and data-plane penalties across high-concurrency workflows, its design exhibits certain architectural boundaries that open up avenues for future evolution.

On the control plane, our Stochastic Sandbox Prefetcher relies on a first-order Markov Chain model, assuming that an agent's sandbox transitions satisfy history-independence. While highly optimal for predictable, graph-structured pipelines, this modeling may suffer

from diminished prediction accuracy when handling open-ended, chaotic agent workflows with ultra-long horizons. To capture these complex, non-linear long-context sandbox-transition patterns, we aim to extend the SDG framework by incorporating lightweight, online graph neural networks (GNNs) in future iterations.

On the data plane, our Out-of-Band Data Transmission relies on memory-mapped files (`mmap`), which restricts the co-location of the Agent Engine and the physical sandboxes within the same host OS boundary. Crucially, such local co-location is fundamentally practical and widely embraced in production serving clusters—typically realized via Kubernetes Pod shared IPC boundaries or container Sidecar patterns—comfortably satisfying the topology requirements of mainstream multi-tenant deployments. Nevertheless, to transcend single-node physical boundaries and support large-scale, cross-node distributed clusters, we plan to explore RDMA-assisted zero-copy transmission structures, expanding our out-of-band data plane into generalized disaggregated data center infrastructures.

7 RELATED WORK

Lightweight Sandbox Runtimes. The systems community has actively explored lightweight isolation mechanisms to mitigate the physical instantiation cost of serverless execution environments. Notable advancements include micro-virtual machines (MicroVMs) like Firecracker [1] and RunD [16], WebAssembly (Wasm) [31] runtimes, and process-level snapshotting/forking frameworks such as FaaSnap [5] and TrEnv-X [11]. These infrastructure-level systems focus on minimizing localized host setup or exploiting hardware-assisted remote memory pools (e.g., CXL/RDMA) to share and reuse physical sandboxes across tenants.

Crucially, these low-layer container optimizations are entirely orthogonal to Laplace. While microVM snapshots or repurposable sandboxes successfully compress the physical infrastructure boot time to milliseconds, application-layer handshakes (such as Model Context Protocol discovery hooks) and environment attachment bottlenecks still natively linger on the critical runtime path. Laplace operates at a higher, application-perceptive orchestration layer; it complements these physical speedups by exploiting a distinct temporal dimension—overlapping control-plane scheduling with streaming token generation—to fully mask, rather than compress, the intrinsic readiness latency.

Serverless Predictive Prewarming. Predictive prewarming is a widely embraced technique in standard serverless frameworks to eradicate the notorious tail-latency penalties of tenant cold starts. State-of-the-art prefetching daemons (e.g., Mitosis [32] and IceBreaker [22]) primarily depend on historical time-series analytics, statistical invocation frequency histograms, or temporal correlation clustering to predictively prepare idle container runtimes.

However, these classical prewarming paradigms inherently assume that incoming requests follow independent and identically distributed arrival models or deterministic time-triggered patterns. This core assumption breaks down completely under emerging LLM Agent workloads. In multi-turn autonomous agent reasoning loops, tool invocations are dynamically, autonomously, and non-linearly determined by the LLM's fluid context trajectory, presenting complex execution dependency horizons. Laplace bridges this gap by co-designing the serving infrastructure with explicit agent behavioral characteristics, introducing a stochastic Markovian predictive framework built over a Sandbox Dependency Graph (SDG) to model real-time autonomous state transitions.

LLM Serving and Agent Orchestration. Accelerating the end-to-end execution of Large Language Models has driven extensive research across the AI systems spectrum. On the model-serving boundary, mainstream runtimes such as vLLM [13] and SGLang [38] optimize GPU-internal kernel execution, memory caching via PagedAttention, and automated speculative decoding. Parallel to this, application-level agent orchestration platforms like AgentScope [8] and AutoGen [33] provide modular abstractions for programming multi-turn autonomous multi-agent teams.

Nevertheless, existing LLM serving engines fundamentally treat model execution as a self-contained GPU computing unit, entirely oblivious to the physical host-plane environment friction when interacting with external tools. Conversely, high-level agent frameworks lack system-level visibility into cloud-native host topologies, resulting in uncoordinated data transfer and execution stalls. Laplace operates as a runtime middleware layer that bridges this gap, enabling co-optimized out-of-band control-plane predictive routing and shared-memory data transport across heterogeneous multi-tenant host execution boundaries.

8 CONCLUSIONS

We present Laplace, a proactive execution runtime for LLM-based agent systems that reduces tail latency and resource inefficiency in multi-tenant environments. By analyzing the end-to-end agent execution loop, we find that inefficiencies primarily arise from rigid dependencies among reasoning, sandbox initialization, and environment execution. Laplace addresses this through a unified execution design that enables temporal overlap between LLM execution and sandbox setup, while also anticipating future sandbox needs across workflow steps to reduce cross-step cold-start overheads.

We implemented Laplace on top of AgentScope [8] and evaluated it under highly concurrent multi-turn workloads. Results show up to a 2.9× reduction in P99 latency and 45.9% lower peak memory usage, with a 97.9% prewarming hit rate. These results demonstrate that exploiting temporal overlap across the agent execution loop is an effective and general mechanism for improving efficiency in large-scale agent deployments.

REFERENCES

[1] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX symposium on networked systems design and implementation (NSDI 20)*. 419–434.

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 117–134.

[3] Amazon Web Services. 2026. *Amazon Bedrock AgentCore- AWS.* https://aws.amazon.com/bedrock/agentcore/ Retrieved June, 2026.

[4] Anthropic PBC. 2024. *Introducing the Model Context Protocol.* https://www.anthropic.com/news/model-context-protocol Retrieved June, 2026.

[5] Lixiang Ao, George Porter, and Geoffrey M Voelker. 2022. Faasnap: Faas made fast using snapshot-based vms. In *Proceedings of the Seventeenth European Conference on Computer Systems*. 730–746.

[6] Datalayer. 2026. *Jupyter MCP Server.* https://github.com/datalayer/jupyter-mcp-server Retrieved June, 2026.

[7] Dong Du, Tianyi Yu, Yubin Xia, Binyu Zang, Guanglu Yan, Chenggang Qin, Qixuan Wu, and Haibo Chen. 2020. Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 467–481.

[8] Dawei Gao, Zitao Li, Xuchen Pan, Weirui Kuang, Zhijian Ma, Bingchen Qian, Fei Wei, Wenhao Zhang, Yuexiang Xie, Daoyuan Chen, et al. 2024. Agentscope: A flexible yet robust multi-agent platform. *arXiv preprint arXiv:2402.14034* (2024).

[9] GitHub. 2026. *MCP Registry*. https://github.com/mcp Retrieved June, 2026.

[10] Google Cloud. 2026. *Gemini Enterprise Agent Platform*. https://docs.cloud.google.com/gemini-enterprise-agent-platform/scale/sandbox Retrieved June, 2026.

[11] Jialiang Huang, Teng Ma, Zheng Liu, Sixing Lin, Kang Chen, Jinlei Jiang, Xia Liao, Yingdi Shan, Yongwei Wu, Ning Zhang, Mengting Lu, Tao Ma, Haifeng Gong, and Mingxing Zhang. 2026. TrEnv-X: Transparently Share Serverless Execution Environments Across Different Functions and Nodes. *ACM Trans. Comput. Syst.* (March 2026). https://doi.org/10.1145/3805475 Just Accepted.

[12] Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov. 2017. Bag of Tricks for Efficient Text Classification. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*. Association for Computational Linguistics, 427–431.

[13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.

[14] Langchain-ai. 2026. *LangGraph: Agent Orchestration Framework for Reliable AI Agents*. https://www.langchain.com/langgraph Retrieved June, 2026.

[15] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. 2024. A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth* 1, 1 (2024), 9.

[16] Zijun Li, Jiagan Cheng, Quan Chen, Eryu Guan, Zizheng Bian, Yi Tao, Bin Zha, Qiang Wang, Weidong Han, and Minyi Guo. 2022. {RunD}: A lightweight secure container runtime for high-density deployment and high-concurrency startup in serverless computing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 53–68.

[17] Kai Mei, Xi Zhu, Wujiang Xu, Mingyu Jin, Wenyue Hua, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. [n. d.]. AIOS: LLM Agent Operating System. In *Second Conference on Language Modeling*.

[18] Microsoft. 2026. *Azure Container Apps Sandboxes*. https://sandboxes.azure.com/ Retrieved June, 2026.

[19] Microsoft. 2026. *Playwright MCP*. https://github.com/microsoft/playwright#playwright-mcp Retrieved June, 2026.

[20] Neo4j. 2026. *Neo4j official MCP Server*. https://github.com/neo4j/mcp Retrieved June, 2026.

[21] Carlo Puliafito, Claudio Cicconetti, Marco Conti, Enzo Mingozzi, and Andrea Passarella. 2022. Stateless or stateful FaaS? I'll take both!. In *2022 IEEE International Conference on Smart Computing (SMARTCOMP)*. IEEE, 62–69.

[22] Rohan Basu Roy, Tirthak Patel, and Devesh Tiwari. 2022. Icebreaker: Warming serverless functions better with heterogeneity. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 753–767.

[23] sentence-transformers. 2026. *all-MiniLM-L6-v2*. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 Retrieved June, 2026.

[24] Mohammad Shahrad, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX annual technical conference (USENIX ATC 20)*. 205–218.

[25] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2024. HybridFlow: A Flexible and Efficient RLHF Framework. *arXiv preprint arXiv: 2409.19256* (2024).

[26] Karen Sparck Jones. 1972. A statistical interpretation of term specificity and its application in retrieval. *Journal of documentation* 28, 1 (1972), 11–21.

[27] Jovan Stojkovic, Tianyin Xu, Hubertus Franke, and Josep Torrellas. 2023. Mxfaas: Resource sharing in serverless environments for parallelism and efficiency. In *Proceedings of the 50th annual international symposium on computer architecture*. 1–15.

[28] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. 2025. Towards end-to-end optimization of llm-based applications with ayo. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1302–1316.

[29] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345.

[30] Zhenting Wang, Qi Chang, Hemani Patel, Shashank Biju, Cheng-En Wu, Quan Liu, Aolin Ding, Alireza Rezazadeh, Ankit Shah, Yujia Bao, et al. 2025. Mcp-bench: Benchmarking tool-using llm agents with complex real-world tasks via mcp servers. *arXiv preprint arXiv:2508.20453* (2025).

[31] WebAssembly. 2026. *WebAssembly*. https://webassembly.org/ Retrieved June, 2026.

[32] Xingda Wei, Fangming Lu, Tianxia Wang, Jinyu Gu, Yuhan Yang, Rong Chen, and Haibo Chen. 2023. No provisioned concurrency: Fast {RDMA-codesigned} remote fork for serverless computing. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 497–517.

[33] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2023. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155* (2023).

[34] Chuhao Xu, Yiyu Liu, Zijun Li, Quan Chen, Han Zhao, Deze Zeng, Qian Peng, Xueqi Wu, Haifeng Zhao, Senbo Fu, et al. 2024. Faasmem: Improving memory efficiency of serverless computing with memory pool architecture. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 331–348.

[35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629* (2022).

[36] Hanfei Yu, Rohan Basu Roy, Christian Fontenot, Devesh Tiwari, Jian Li, Hong Zhang, Hao Wang, and Seung-Jong Park. 2024. Rainbowcake: Mitigating cold-starts in serverless with layer-wise container caching and sharing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 335–350.

[37] Yihui Zhang, Han Shen, Renyu Yang, Di Tian, Yuxi Luo, Menghao Zhang, Li Li, Chunming Hu, Tianyu Wo, Chengru Song, et al. 2025. Cauchy: A Cost-Efficient LLM Serving System through Adaptive Heterogeneous Deployment. In *Proceedings of the 2025 ACM Symposium on Cloud Computing*. 881–893.

[38] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.

[39] Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. 2025. slime: An LLM post-training framework for RL Scaling. https://github.com/THUDM/slime. GitHub repository. Corresponding author: Xin Lv.